Project Report on

FAKE NEWS DETECTION USING MACHINE LEARNING

Submitted in partial fulfilment of the requirement for the award of the Degree of Bachelor of Computer Application (BCA)

SUBMITTED BY

MAHADEV SWAMY

REG. No. U27XK23S0054

Under the Guidance of

Asst. Prof. NAMRATA PATIL



Department of Computer Science

CHANNABASAMMA COLLEGE OF BCA & B.COM BIDAR

2025-2026

Certificate

This is to certify that the project entitled "FAKE NEWS DETECTION USING MACHINE LEARNING" submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Applications (BCA) to the Registrar (Evaluation), Bidar University, through CHANNABASAMMA COLLEGE OF BCA & B.COM, BIDAR, is a bonafide work carried out by MAHADEV SWAMY bearing REG. NO. U27XK23S0054 under my guidance.

It is further certified that this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Signature of the Guide

Asst. Prof. NAMRATA PATIL

CHANNABASAMMA COLLEGE OF BCA & B.COM

BIDAR-585401



Certificate

This is to certify that the live project entitled "**FAKE NEWS DETECTION USING MACHINE LEARNING**" submitted by **MAHADEV SWAMY**, bearing **REG. NO. U27XK23S0054**, to **BIDAR UNIVERSITY** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Applications (BCA), is a bonafide record of the work carried out under my guidance in collaboration with the Department of Computer Science during the academic year **2025–2026**.

PROJECT GUIDE	HOD	PRINCIPAL
Asst. Prof. NAMRATA PATIL	Asst. Prof. DATTATRI	Dr. SATISH PRATAPUR

Examiners

1.

2.

ACKNOWLEDGEMENT

I would like to thank the management of Sri SHARANABASAVESHWAR VIDYA VARDHAK SANGHA for providing facilities to complete my project successfully.

I extend my deep gratitude to the Principal Dr. SATISH PRATAPUR of CHANNABASAMMA COLLEGE OF BCA & B.COM, BIDAR, for kind support extended during the project.

I wish to thank Asst. Prof. DATTATRI, Head of the Department for his kind co-operation during my project.

I wish to express my sincere gratitude to my Project Guide Asst. Prof. NAMRATA PATIL in Computer Science Department for her continuous support, encouragement and valuable guidance during the project work.

I was benefited a lot from her Project Developing Skill, constructive suggestions, dedication, discipline and efforts to accomplish timely completion of my project. I shall always remain indebted to her.

I wish to thank, other staff members of the Department Of Computer Science for their valuable guidance, suggestions and encouragement which enabled me to complete the project.

Finally, and most importantly I want to thank my Parents and all my friends for their kind cooperation throughout my project work.

MAHADEV SWAMY

REG.NO. U27XK23S0054

Declaration

I hereby declare that the project report entitled "FAKE NEWS DETECTION USING MACHINE LEARNING" is a bonafide work carried out by me in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Applications (BCA) under the guidance of Asst. Prof. NAMRATA PATIL.

I further declare that the matter embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Signature of the student

MAHADEV SWAMY

REG No. U27XK23S0054

CONTENTS

1. INTRODUCTIONS

1.1 Abstract

2. PROBLEM STATEMENT

2.1 Existing system

2.2 Proposed system

3. SYSTEM ANALYSIS AND DESIGN

3.1 System study

3.2 Feasibility study

3.3 Requirement specification

3.4 Software development cycle

4. SYSTEM DESIGN

4.1 Function oriented design

4.2 Use case diagram

4.3 ERD & DFD

5. DATABASE DESIGN

5.1 Introduction

6. SYSTEM REQUIREMENTS

6.1 Software requirement

6.2 Hardware requirement

7. IMPLEMENTATION

7.1 Web development introduction

7.2 The terminologies in web development

7.3 Silent features of web development

8. INTEGRATION

9. TESTING

10. SCREENSHOTS

11. CONCLUSION

12. REFERENCES

Chapter - 1

INTRODUCTION

1.1 ABSTRACT

In recent years, due to the booming development of online social networks, fake news for various commercial and political purposes has been appearing in large numbers and widespread in the online world. With deceptive words, online social network users can get infected by these online fake news easily, which has brought about tremendous effects on the offline society already.

An important goal in improving the trustworthiness of information in online social networks is to identify the fake news timely. This paper aims at investigating the principles, methodologies and algorithms for detecting fake news articles, creators and subjects from online social networks and evaluating the corresponding performance. Information preciseness on Internet, especially on social media, is an increasingly important concern, but web-scale data hampers, ability to identify, evaluate and correct such data, or so called "fake news," present in these platforms. In this paper, we propose a method for "fake news" detection and ways to apply it on Facebook, one of the most popular online social media platforms.

This method uses Naive Bayes classification model to predict whether a post on Facebook will be labeled as real or fake. The results may be improved by applying several techniques that are discussed in the paper. Received results suggest, that fake news detection problem can be addressed with machine learning methods.

Chapter - 2

PROBLEM STATEMENT

2.1 EXISTING SYSTEM

"Fake News Detection using Machine Learning" generally refers to traditional manual fact-checking and early-stage automated text classification methods. These systems primarily focus on the linguistic content of news articles rather than the context or source credibility.

Core Components of Existing Systems

- **Manual Fact-Checking:** Traditional detection relies on human experts who verify claims manually. This method is highly accurate but not scalable for the millions of articles published every minute.
- **Simple Textual Analysis:** Early automated systems use **n-gram analysis** (counting sequences of words) and shallow **Parts-of-Speech (POS) tagging** to identify deceptive patterns. However, these often fail to account for complex context and linguistic nuance.
- **Standard ML Classifiers:** Most existing models employ supervised learning algorithms to perform **binary classification** (Real vs. Fake). Common algorithms include:
- **Naive Bayes:** Often used as a baseline for text processing due to its simplicity and efficiency.
- **Logistic Regression:** Frequently used for two-class problems, achieving high accuracy in controlled environments.
- **Support Vector Machines (SVM):** Used to create a hyperplane that separates genuine data from fake news.
- **Random Forest:** An ensemble method that reduces overfitting by averaging multiple decision trees.

2.2 PROPOSED SYSTEM

The proposed system aims to automatically detect and classify news articles as **real or fake** using machine learning techniques. It is designed to overcome the limitations of manual verification by providing fast, scalable, and accurate results.

1. System Overview

The system takes news content (text input such as headlines or full articles) from users and processes it through a trained machine learning model to predict whether the news is **genuine or fake**.

2. Architecture of the Proposed System

The system consists of the following major components:

a) Data Collection

- Gather datasets containing labeled news articles (real and fake).
- Sources may include publicly available datasets like Kaggle or news APIs.

b) Data Preprocessing

- Remove noise such as punctuation, stopwords, and special characters.

c) Feature Extraction

- Convert textual data into numerical format using techniques like:
- TF-IDF (Term Frequency–Inverse Document Frequency)

d) Model Training

- Apply machine learning algorithms such as:
- Decision Tree
- Random Forest

Chapter - 3

SYSTEM ANALYSIS AND DESIGN

3.1 SYSTEM STUDY

The system study phase focuses on understanding the existing environment, identifying problems, and defining requirements for the proposed Fake News Detection system. With the rapid spread of misinformation through digital platforms, there is a need for an automated system that can classify news as real or fake using machine learning techniques.

2. Existing System

Currently, fake news detection is mostly:

- Manual verification by fact-checkers
- Based on user reports or complaints
- Dependent on third-party fact-checking websites

Limitations of Existing System

- Time-consuming and slow process
- Not scalable for large volumes of data
- Human bias and inconsistency
- Delayed response to viral fake news

3. Proposed System

The proposed system uses machine learning algorithms to automatically classify news articles as fake or real.

Key Features

- Automatic text classification
- Real-time detection capability

3.2 FEASIBILITY STUDY

The feasibility study evaluates whether the proposed Fake News Detection system is practical and viable. It analyzes different aspects such as technical, economic, operational, and other feasibility factors to determine whether the system can be successfully developed and implemented.

- **Technical Feasibility**

This examines whether the required technology and resources are available.

- The system uses Machine Learning and Natural Language Processing (NLP) techniques.
- Technologies like Python, Scikit-learn, Pandas, NumPy, and NLTK are widely available and open-source.
- Datasets for fake news detection are publicly available (e.g., Kaggle datasets).

- **Economic Feasibility**

This evaluates the cost-benefit aspect of the project.

- Development cost is low since it uses free and open-source tools.
- No need for expensive infrastructure or paid software.
- Maintenance cost is minimal.

- **Operational Feasibility**

This determines whether the system will be accepted and used effectively.

- The system provides a simple interface for users to input news.
- Requires minimal user training.

- **Legal Feasibility**

This checks whether the system complies with laws and regulations.

- The system uses publicly available datasets.
- Must ensure data privacy and ethical use of information.

- **Schedule Feasibility**

This evaluates whether the project can be completed within the given time.

- The project can typically be completed in 2–4 months depending on complexity.

3.3 REQUIREMENT SPECIFICATION

Requirement Specification defines the hardware, software, functional, and non-functional requirements needed for developing and implementing the project “Fake News Detection using Machine Learning.” It specifies the resources and functionalities required for the successful execution of the system.

1. Hardware Requirements

The following hardware components are required for developing and running the system:

• Processor : Intel Core i3/i5 or higher
• RAM : Minimum 4 GB
• Hard Disk : 250 GB or above
• System Type : 64-bit Computer
• Input Devices : Keyboard and Mouse
• Output Device : Monitor

2. Software Requirements

The following software technologies and tools are required for system development:

• Operating System : Windows 10/11 or Linux
• Programming Language : Python
• IDE/Editor : VS Code, PyCharm, or Jupyter Notebook
• Database : CSV Dataset / MySQL (optional)
• Libraries Used :
• Pandas
• NumPy
• Scikit-learn
• NLTK
• Matplotlib
• Flask (for web application)

3. Functional Requirements

Functional requirements describe the main operations and functionalities of the system.

• The system should collect news datasets containing real and fake news articles.
• The system should perform feature extraction using techniques such as TF-IDF.
• The system should train machine learning models using labeled news datasets.
• The system should classify news articles as Real or Fake.
• The system should display prediction results accurately to the user.
• The system should allow users to enter news content for prediction.

5. Dataset Requirements

• The system requires datasets containing real and fake news articles.
• Datasets may be collected from Kaggle or other publicly available sources.
• The dataset should contain labeled data for training and testing the machine learning models.

3.4 SOFTWARE DEVELOPMENT CYCLE

The software development life cycle (SDLC) of a **Fake News Detection using Machine Learning** system is a structured process that ensures the effective design, development, and deployment of the application. It begins with planning and requirement analysis, where the problem of identifying fake news is clearly defined and system requirements are gathered. In the design phase, the architecture is created, including data processing modules and machine learning components. The development phase involves collecting and preprocessing datasets, extracting features using natural language processing techniques, and training classification models to distinguish between fake and real news.

1. Requirement Analysis

Understand what the system must do.

Functional Requirements:

- Input news text or URL
- Classify news as **Fake** or **Real**

Non-Functional Requirements:

- High accuracy and performance
- Scalability for large datasets

2. System Design

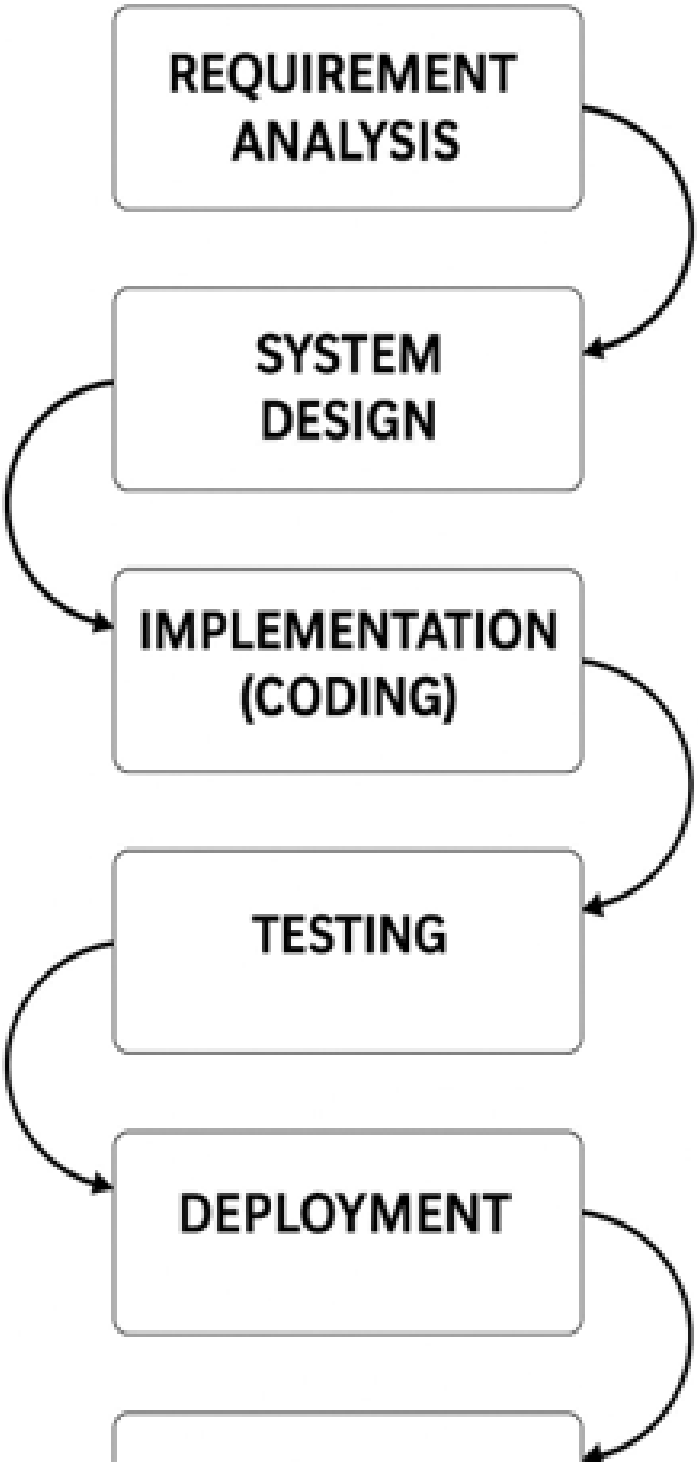
Design the architecture of the system.

High-Level Design:

- Frontend (UI for user input)
- Backend (API + ML model)

Low-Level Design:

- Data preprocessing module
- Feature extraction (TF-IDF, NLP techniques)
- Model selection (e.g., Logistic Regression, Naive Bayes, or Deep Learning)


REQUIREMENT ANALYSIS
SYSTEM DESIGN
IMPLEMENTATION (CODING)
TESTING
DEPLOYMENT

Chapter - 4

SYSTEM DESIGN

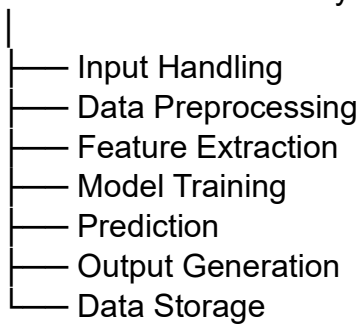
4.1 FUNCTION ORIENTED DESIGN

1. Overview

Function-Oriented Design focuses on breaking the system into smaller functional modules based on the operations it performs. Each function handles a specific task, and all functions work together to achieve the system goal: detecting whether news is real or fake.

2. Top-Level Functional Decomposition

The system can be divided into the following main functions:

Fake News Detection System


3. Functional Hierarchy (Detailed Breakdown)

a. Input Handling Function

- Accepts:
  - News text
  - News URL
- Sub-functions:
  - Validate input
  - Fetch content (if URL)

b. Data Preprocessing Function

- Cleans and prepares raw data

Sub-functions:

- Text normalization (lowercase)
- Remove punctuation
- Remove stopwords
- Stemming / Lemmatization
- Tokenization

c. Feature Extraction Function

- Converts text into numerical format

Sub-functions:

- Apply TF-IDF Vectorizer
- Bag-of-Words conversion
- Feature selection

d. Model Training Function

- Trains machine learning models

Sub-functions:

- Load dataset
- Split data (train/test)
- Train model (e.g., Logistic Regression, Naive Bayes)
- Evaluate model accuracy

e. Prediction Function

- Predicts whether news is fake or real

Sub-functions:

- Load trained model
- Transform input data
- Generate prediction
- Calculate confidence score

f. Generation Function

- Displays results to user

Sub-functions:

- Show label (Fake / Real)
- Show probability (optional)
- Format output

g. Data Storage Function

- Stores system data

Sub-functions:

- Save trained models
- Store user queries

4. Data Flow in Functional Design

User Input
↓
Input Handling
↓
Preprocessing
↓
Feature Extraction
↓
Prediction (Model)
↓
Output Display
↓
Data Storage

5. Structure Chart (Text Representation)

Main()
│
├── get_input()
├── preprocess_text()
├── extract_features()
├── load_model()
├── predict_news()
├── display_result()
└── save_result()

4.2 USE CASE DIAGRAM

1. Overview

A Use Case Diagram represents how different users (actors) interact with the system. It shows the system's functionality from the user's perspective.

In this project, the diagram illustrates how users input news and receive predictions.

2. Actors in the System

- **1. User**
  - Main actor
  - Can:
    - Enter news text
    - View prediction result

3. Main Use Cases

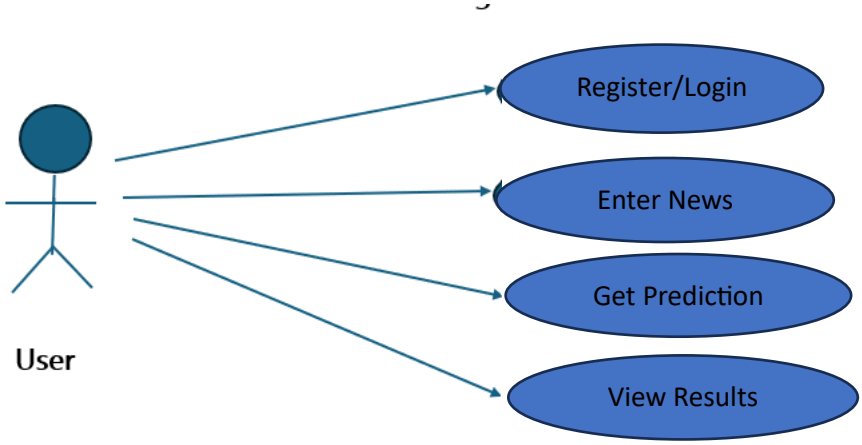
For User:

- Enter News Text
- Upload News
- Analyze News
- View Result (Fake/Real)
- Train Model

4. Use Case Relationships

- **Include Relationship:**
  - "Analyze News" includes:
    - Preprocessing
    - Feature Extraction
    - Prediction
- **Extend Relationship**:
  - "View Result" may extend to:
    - View confidence score
    - View detailed analysis

5. Use Case Diagram



4.3 ERD & DFD

Overview

An ERD shows how data is stored in the system and how different entities (tables) are related.

- **Main Entities**

1. User

- user_id (PK)
- name
- email

2. News_Input

- input_id (PK)
- content
- url
- user_id (FK)

3. Prediction

- prediction_id (PK)
- result (Fake/Real)
- confidence_score
- input_id (FK)

4. Model

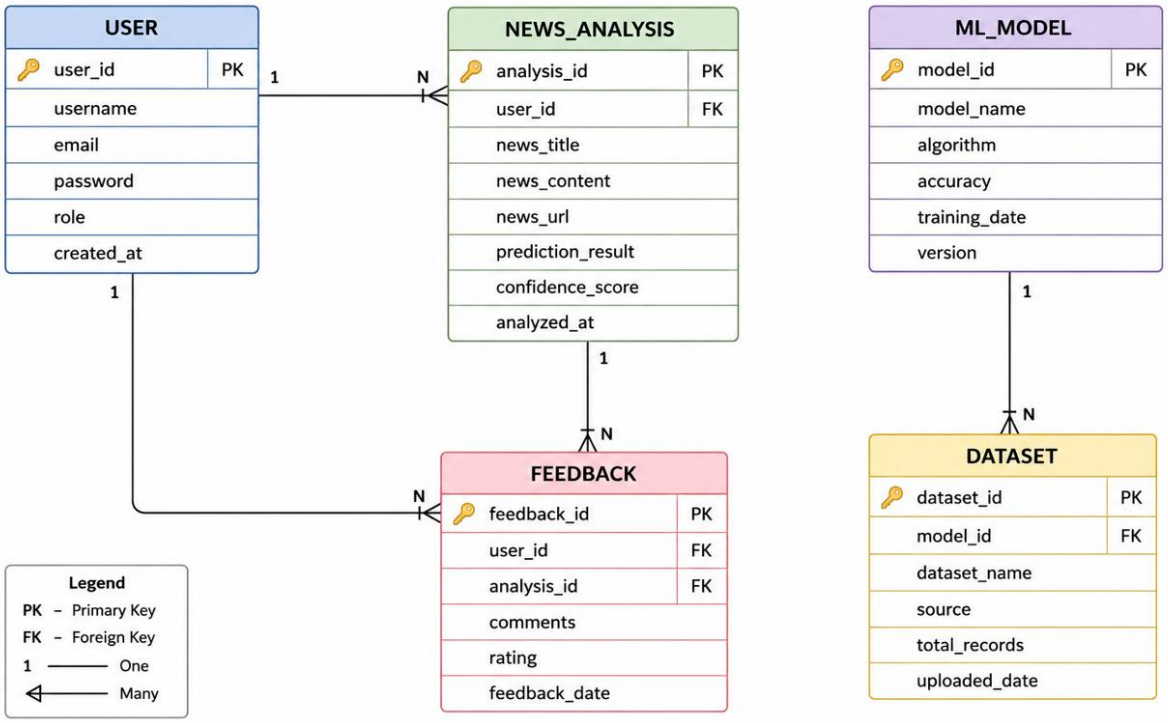
- model_id (PK)
- model_name
- accuracy
- version

5. Dataset

- dataset_id (PK)
- dataset_name
- source
- size

- **Relationships**
  - One **User** → many **News Inputs**
  - One **News Input** → one **Prediction**
  - One **Model** → many **Predictions**
  - One **Dataset** → used by **Model**

- **ERD Diagram (Figure)**



- **ERD Explanation**
  - Tracks user activity and predictions
  - Stores ML model performance
  - Maintains dataset details
  - Helps in database design

2. Data Flow Diagram (DFD)

- **Overview**

A DFD shows how data flows through the system from input to output.

- **Components**

External Entity:

- User

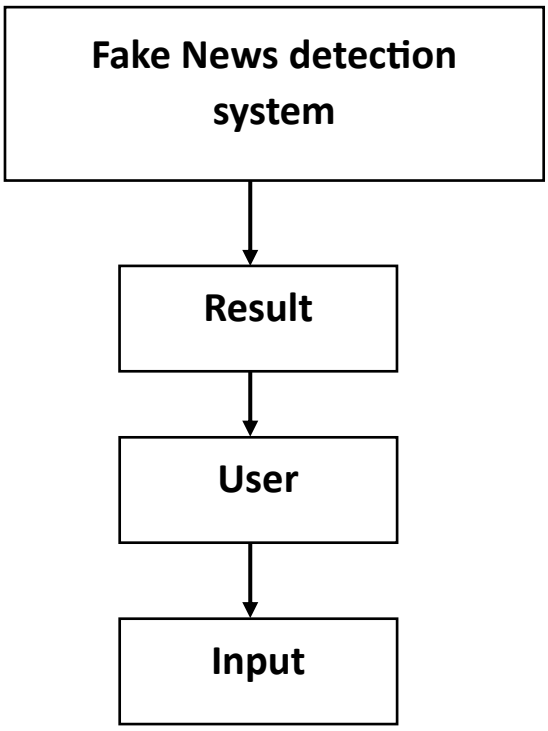
Processes:

- Input Processing
- Text Preprocessing
- Feature Extraction
- Prediction Engine

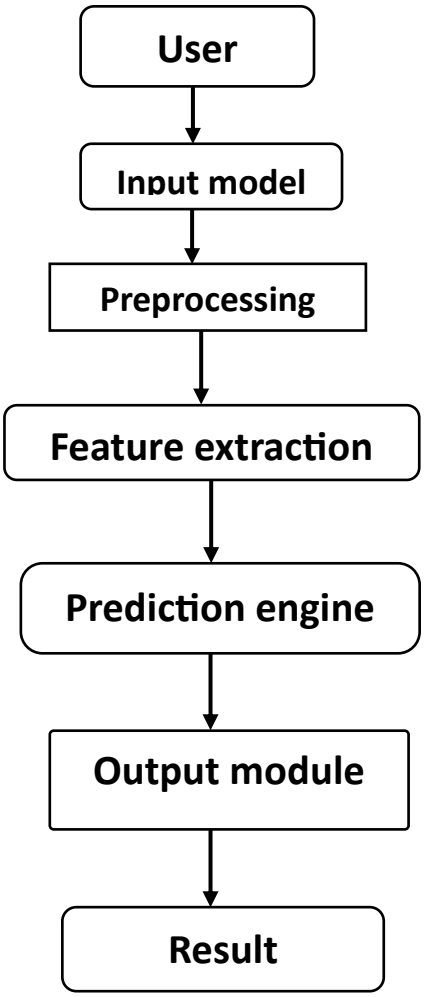
Data Stores:

- Dataset
- Model Storage
- Prediction Records

- **DFD Level 0 (Context Diagram)**



- **DFD Level 1 (Detailed Diagram)**



Data Stores:
[Dataset] ----> [Prediction Engine]
[Model] ----> [Prediction Engine]
[Logs] <---- [Output Module]

- **DFD Explanation**

Step-by-step flow:

1. User inputs news text/URL
2. System preprocesses the text
3. Features are extracted
4. ML model predicts result
5. Output is shown to user
6. Result stored in database

Chapter – 5

DATABASE DESIGN

5.1 INTRODUCTION

The database design for the *Fake News Detection using Machine Learning* project plays a crucial role in efficiently storing, managing, and retrieving large volumes of news data and model-related information. The system requires a well-structured database to handle datasets containing news articles, user inputs, prediction results, and model metadata.

The primary objective of the database is to support the machine learning pipeline by organizing both structured and unstructured data. It stores essential information such as news content (title, body, source), labels (real or fake), preprocessing outputs, and classification results generated by the trained model. Additionally, the database maintains user interaction data, which can be used for further analysis and system improvement.

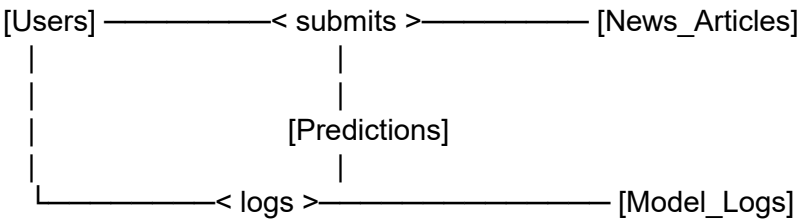
A relational database design is typically adopted, where data is organized into multiple tables such as Users, News Articles, Predictions, and Model Logs. Each table is connected through relationships using keys to ensure data integrity and avoid redundancy. Proper indexing and normalization techniques are applied to enhance performance and scalability.

The database also supports real-time querying, allowing the system to quickly fetch and process user-submitted news content for prediction. It ensures data consistency, security, and efficient storage management, which are essential for building a reliable and scalable fake news detection system.

Overall, the database design acts as the backbone of the system, enabling seamless integration between data collection, machine learning processing, and result delivery.

1. ER Diagram (Entity Relationship Diagram)

Entities & Relationships



Explanation

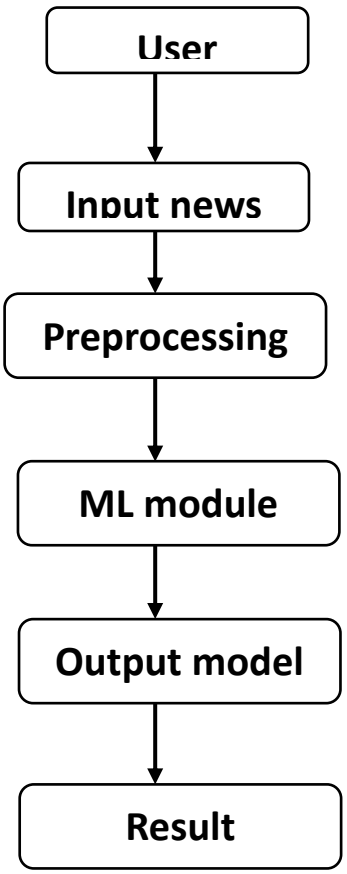
- A **User** submits news articles
- Each **News Article** gets classified
- The result is stored in **Predictions**

2. Data Flow Diagram (DFD)

➢ Level 0 (Context Diagram)

- User ——> Fake News Detection System ——> Prediction Result

➢ Level 1 DFD



Chapter - 5

SYSTEM REQUIREMENTS

6.1 SOFTWARE REQUIREMENT

❖ **Front End:** HTML, CSS, JavaScript

❖ **Back End:** Python (Flask / Django)

❖ **Machine Learning Libraries:** Scikit-learn, Pandas, NumPy

❖ **NLP Tools:** NLTK, spacy

❖ **IDE:** Visual Studio Code

❖ **Operating System:** Windows 10, Windows 11

6.2 Hardware Requirements:-

❖ **Processor:** Intel i3 or higher

❖ **RAM:** Minimum 4 GB (8 GB recommended)

❖ **Storage:** Minimum 10 GB free disk space

❖ **System Type:** 64-bit Computer

❖ **Input Devices:** Keyboard, Mouse

❖ **Output Devices:** Monitor

❖ **Internet Connection:** Required for dataset download and API usage

Chapter - 6

IMPLEMENTATION

7.1 WEB DEVELOPMENT INTRODUCTION

Web development in the *Fake News Detection using Machine Learning* project involves building a web-based application that allows users to input news content and receive predictions about its authenticity. It ensures smooth interaction between the user interface and the machine learning model. It typically includes three main layers:

- **Frontend (Client-Side):** Interface where users interact with the system, built using HTML, CSS, and JavaScript to input news text or URLs and display results.
- **Backend (Server-Side):** Handles server logic, processes user requests, communicates with the machine learning model, and manages APIs using technologies like Node.js, Flask, or Django.
- **Database:** A structured data storage system like MySQL or MongoDB used to store user data, news articles, and prediction results for future reference.

7.2 THE TERMINOLOGIES IN WEB DEVELOPMENT

1.Frontend (Client-Side): The part of the application that users interact with, developed using HTML, CSS, and JavaScript.

2.Backend (Server-Side): Handles server logic, processes user requests, communicates with the machine learning model, and manages APIs using technologies like Node.js, Flask, or Django.

3. Full-Stack Development: Full-stack development refers to the combination of frontend and backend development. A full-stack developer works on the entire application stack, from user interfaces to server logic and databases. The Fake News Detection project is a full-stack application as it includes both the frontend (HTML/CSS/JavaScript) and backend (Node.js or Python) along with database integration.

4. HTML (Hypertext Markup Language): HTML is the standard markup language used to create web pages. It defines the structure of content. In this project, HTML is used to design input forms where users can enter news text or URLs for detection.

5. CSS (Cascading Style Sheets): CSS is used to control the appearance and layout of HTML elements. It ensures the application is visually appealing and user-friendly. In this project, CSS is used to style the user interface and result display pages.

6. Express.js: Express.js is a lightweight web framework for Node.js that simplifies routing and middleware handling. In this project, Express is used to handle user requests, process input data, and connect with the machine learning model.

7. Node.js: Node.js is a JavaScript runtime environment that allows JavaScript to run on the server side. It is used to manage server operations, handle requests, and interact with the database in this project.

.

7.3 SILENT FEATURES OF WEB DEVELOPMENT

- **User-Friendly Interface:**
  The web application provides a simple and intuitive interface where users can easily input news content or URLs to check authenticity.
- **Real-Time News Analysis:**
  The system processes user input instantly and displays whether the news is **fake or real** using trained machine learning models.
- **Responsive Web Design:**
  The application is designed to work smoothly across devices such as desktops, tablets, and mobile phones.
- **Integration with Machine Learning Model:**
  The backend is connected with ML algorithms (like Naive Bayes, Logistic Regression, etc.) to analyze and classify news data effectively.
- **Secure Data Handling:**
  Ensures user inputs and data are handled securely using proper validation and backend processing.
- **Database Connectivity:**
  Stores user queries, prediction results, and datasets for future analysis and system improvement.
- **Fast Processing Speed:**
  Optimized backend ensures quick response time for predictions and minimal delay.
- **Scalability:**
  The web application can handle multiple users simultaneously and can be expanded with additional features in the future.
- **API Integration:**
  Can be integrated with external APIs (news sources, fact-checking services) to enhance accuracy.
- **Visualization of Results:**
  Displays prediction results clearly, sometimes with probability scores or graphs for better understanding.

Chapter - 7

CODING

About.html :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About Verity</title>
  <link rel="stylesheet" href="/static/CSS/style.css">
</head>
<body>
  <div class="noise"></div>
  <header>
    <div class="logo">
      <span class="logo-icon">📰</span>
      <div>
        <div class="logo-text">VERITY</div>
        <div class="logo-subtitle">About</div>
      </div>
    </div>
    <div class="header-right">
      <a href="/" class="nav-link">Home</a>
    </div>
  </header>
  <main>
    <section class="hero">
      <p class="eyebrow">About the project</p>
      <h1>Fake news detection with a beautiful UI.</h1>
      <p class="subtitle">Verity is a demo frontend for a fake-news detector. It features a modern login/signup flow, a polished analysis dashboard, and persistent history for every user.</p>
    </section>
```

```
    <section class="card">
      <h2>What it includes</h2>
      <ul class="about-list">
        <li>Interactive login and signup pages</li>
        <li>Clean news analysis dashboard</li>
        <li>History details stored locally</li>
        <li>Responsive, dark-themed layout</li>
      </ul>
    </section>
  </main>
  <footer>Built for the Verity fake news detection demo.</footer>
</body>
</html>
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Verity — Fake News Detector</title>
  <link rel="stylesheet" href="/static/CSS/style.css">
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" cross origin>
  <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;700&family=Spac
e+Mono:wght@400;700&display=swap" rel="stylesheet">
</head>
<body>
  <div class="noise"></div>
  <header>
    <div class="logo">
      <span class="logo-icon">📰</span>
```

```
      <div>
        <div class="logo-text">VERITY</div>
        <div class="logo-subtitle">Smart news credibility</div>
      </div>
    </div>
    <div class="header-right">
      <div class="user-badge">
        <span id="userGreeting">Hello, Guest</span>
        <button id="logoutBtn" class="btn btn-ghost logout-link">Logout</button>
      </div>
      <div class="model-badge" id="modelStatus">Checking model…</div>
    </div>
  </header>

  <main>
    <section class="hero">
      <p class="eyebrow">Trusted news intelligence</p>
      <h1>Detect <em>fake</em> and <strong>real</strong> news in seconds.</h1>
      <p class="subtitle">Paste any headline or article snippet, then let Verity analyze it with a trained model. Keep your reading history and track every decision.</p>
    </section>

    <section class="card" id="analyzeCard">
      <div class="card-header">
        <div>
          <h2>Analyze news text</h2>
          <p class="subtitle">Type or paste a news story and tap Analyze.</p>
        </div>
        <button id="retrainBtn" class="btn btn-primary retrain-btn">Retrain Model</button>
      </div>

      <label class="field-label" for="newsInput">Enter text</label>
```

```
    <textarea id="newsInput" placeholder="Paste a news headline, paragraph, or social post here..." spellcheck="false"></textarea>

    <div class="input-footer">
      <div class="word-count" id="wordCount">0 words</div>
      <div class="btn-group">
        <button id="clearBtn" class="btn btn-ghost">Clear</button>
        <button id="analyzeBtn" class="btn btn-primary">Analyze</button>
      </div>
    </div>

    <div class="samples">
      <div class="samples-label">Try sample text</div>
      <div class="sample-pills">
        <button type="button" class="pill" data-text="Scientists confirm that climate change is driving extreme weather events worldwide.">Science report</button>
        <button type="button" class="pill" data-text="Celebrity launches new beauty brand after viral skincare controversy.">Celebrity story</button>
        <button type="button" class="pill" data-text="Breaking: election results are being manipulated by secret foreign groups." >Conspiracy claim</button>
      </div>
    </div>
  </section>

  <section id="resultSection" class="result-section" hidden>
    <section class="result-card verdict-card" id="verdictCard" hidden>
      <div class="verdict-header">
        <div class="verdict-icon-wrap" id="verdictIcon">
```

```
        </div>
        <div>
          <div class="verdict-label">Prediction</div>
          <div class="verdict-value" id="verdictLabel">REAL</div>
        </div>
      </div>
```

```
  <div class="confidence-block">
    <div class="conf-labels">
      <span>Real confidence</span>
      <span>Fake confidence</span>
    </div>
    <div class="conf-track">
      <div id="realBar" class="conf-bar real-bar" style="width:0%"></div>
      <div id="fakeBar" class="conf-bar fake-bar" style="width:0%"></div>
    </div>
    <div class="conf-values">
      <span id="realPct">0%</span>
      <span id="fakePct">0%</span>
    </div>
  </div>
  <div class="meta-row">
    <div class="meta-item">
      <div class="meta-label">Confidence</div>
      <div class="meta-value" id="confidenceVal">0%</div>
    </div>
    <div class="meta-item">
      <div class="meta-label">Words</div>
      <div class="meta-value" id="wordCountResult">0</div>
    </div>
    <div class="meta-item">
      <div class="meta-label">Model</div>
      <div class="meta-value" id="modelName">Unknown</div>
    </div>
  </div>
</section>

<section class="card loading-card" id="loadingCard" hidden>
  <div class="spinner"></div>
```

```
      <div>Analyzing text... please wait.</div>
    </section>

    <section class="card error-card" id="errorCard" hidden>
      <span>⚠</span>
      <span id="errorMsg">An error occurred.</span>
    </section>
  </section>

  <section class="card history-card" id="historyCard" hidden>
    <div class="card-header">
      <div>
        <h2>History details</h2>
        <p class="subtitle">Track every analysis and review your recent predictions.</p>
      </div>
      <button id="clearHistoryBtn" class="btn btn-ghost">Clear history</button>
    </div>
    <div id="historyList" class="history-list"></div>
  </section>

  <section class="card how-it-works">
    <h3>How Verity works</h3>
    <div class="steps">
      <div class="step">
        <div class="step-num">01</div>
        <div class="step-text"><strong>Paste your text.</strong> Get started with any news headline, tweet, or paragraph.</div>
      </div>
      <div class="step">
        <div class="step-num">02</div>
        <div class="step-text"><strong>Analyze the result.</strong> Verity classifies whether the text looks fake or real.</div>
      </div>
```

```
        <div class="step">
          <div class="step-num">03</div>
          <div class="step-text"><strong>Review history.</strong> All decisions are saved so you can inspect trends later.</div>
        </div>
      </div>
    </section>
  </main>

  <footer>Verified news decisions saved locally · Login required to use the dashboard.</footer>
  <script>
    if (!localStorage.getItem('verity_session')) {
      window.location.href = '/login';
    }
  </script>
  <script src="/static/js/app.js"></script>
</body>
</html>
```

Login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login | Verity</title>
  <link rel="stylesheet" href="/static/css/style.css">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
</head>
<body class="login-bg">
  <div class="noise"></div>
  <div class="login-container" id="login-container">
```

```
    <div class="login-lock"><i class="fas fa-shield-alt"></i></div>
    <h2 class="welcome-title">Welcome back</h2>
    <p class="subtitle">Log in to access Verity and review your analysis history.</p>

    <form id="loginForm" class="auth-form">
      <div class="input-group">
        <input id="loginUsername" type="text" placeholder="Username"
autocomplete="username" required>
        <i class="fas fa-user"></i>
      </div>
      <div class="input-group">
        <input id="loginPassword" type="password" placeholder="Password"
autocomplete="current-password" required>
        <i class="fas fa-lock"></i>
      </div>
      <div id="formError" class="form-banner error-banner"></div>
      <div id="formSuccess" class="form-banner success-banner"></div>
      <button id="loginBtn" type="button" class="btn btn-primary">Log in</button>
    </form>

    <div class="divider">or</div>
    <div class="signup-link">New here? <a href="/signup">Create an account</a></div>
  </div>
  <script src="/static/js/auth.js"></script>
</body>
</html>
```

Signup.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Sign Up | Verity</title>
  <link rel="stylesheet" href="/static/CSS/style.css">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
</head>
<body class="login-bg">
  <div class="noise"></div>
  <div class="login-container" id="signup-container">
    <div class="login-lock"><i class="fas fa-user-plus"></i></div>
    <h2 class="welcome-title">Create Account</h2>
    <p class="subtitle">Sign up and then return to login to use Verity.</p>

    <form id="signupForm" class="auth-form">
      <div class="input-group">
        <input id="signupUsername" type="text" placeholder="Username" autocomplete="username" required>
        <i class="fas fa-user"></i>
      </div>
      <div class="input-group">
        <input id="signupEmail" type="email" placeholder="Email (optional)" autocomplete="email">
        <i class="fas fa-envelope"></i>
      </div>
      <div class="input-group">
        <input id="signupPassword" type="password" placeholder="Password" autocomplete="new-password" required>
        <i class="fas fa-lock"></i>
      </div>
      <div class="input-group">
        <input id="signupConfirm" type="password" placeholder="Confirm password" autocomplete="new-password" required>
        <i class="fas fa-check"></i>
      </div>
```

```
      <div id="formError" class="form-banner error-banner"></div>
      <div id="formSuccess" class="form-banner success-banner"></div>
      <button id="signupBtn" type="button" class="btn btn-primary">Create account</button>
    </form>

    <div class="divider"></div>
    <div class="signup-link">Already have an account? <a href="/login">Login</a></div>
  </div>
  <script src="/static/js/auth.js"></script>
</body>
</html>
```

App.py :

```
"""
app.py
------
Flask web application for the Fake News Detector.

Routes:
    GET  /          → Main UI page
    GET  /about     → About page
    GET  /login     → Login page
    POST /api/login  → Login endpoint
    POST /api/signup → Signup endpoint
    GET  /api/status → Model status check
    POST /api/predict → Run prediction
    GET  /api/info   → Model metadata
    POST /api/retrain → Retrain model
    GET  /api/retrain/status → Retrain status
"""

import os
import sys
```

```
import threading
import time
from flask import Flask, render_template, request, jsonify, send_from_directory
from flask_cors import CORS

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from src.predict import predict, is_model_loaded, get_model_name, reload_model

# ── App Setup ──────────────────────────────────────────────────────────────────

app = Flask(__name__)
app.secret_key = 'verity_fake_news_detector_2024'
CORS(app)

# In-memory user store (for demo - matches localStorage structure)
# Passwords are stored as hashes to match auth.js
USERS = {
    "demo": {"password": "v1_-542681653", "email": "demo@example.com", "createdAt": 1234567890},
    "admin": {"password": "v1_607284838", "email": "admin@example.com", "createdAt": 1234567890}
}

# Retrain status tracking
_retraining = False
_retraining_result = None
_retraining_start_time = None

# ── Static Routes for HTML Templates ──────────────────────────────────────────

@app.route("/")
def index():
```

```
    """Serve the main UI."""
    return render_template("index.html")

@app.route("/about")
def about():
    """Serve the about page."""
    return render_template("about.html")

@app.route("/login")
def login_page():
    """Serve the login page."""
    return render_template("login.html")

@app.route("/signup")
def signup_page():
    """Serve the signup page."""
    return render_template("signup.html")

# ── API Routes for Authentication ──────────────────────────────────────

@app.route("/api/login", methods=["POST"])
def api_login():
    """API endpoint for login."""
    data = request.get_json()

    if not data:
        return jsonify({"success": False, "error": "Invalid request"}), 400

    username = data.get("username", "").strip()
    password = data.get("password", "").strip()
```

```
    if not username or not password:
        return jsonify({"success": False, "error": "Username and password required"}), 400

    # Hash password to compare with stored hash
    hashed_password = hash_password(password)

    user = USERS.get(username)
    if user and user.get("password") == hashed_password:
        return jsonify({
            "success": True,
            "user": {
                "username": username,
                "email": user.get("email", "")
            }
        })
    else:
        return jsonify({"success": False, "error": "Invalid username or password"}), 401

@app.route("/api/signup", methods=["POST"])
def api_signup():
    """API endpoint for signup."""
    data = request.get_json()

    if not data:
        return jsonify({"success": False, "error": "Invalid request"}), 400

    username = data.get("username", "").strip()
    password = data.get("password", "").strip()

    # Validation
    if not username or not password:
        return jsonify({"success": False, "error": "Username and password required"}), 400
```

```
    if len(username) < 3:
        return jsonify({"success": False, "error": "Username must be at least 3 characters"}),
400

    if len(password) < 6:
        return jsonify({"success": False, "error": "Password must be at least 6 characters"}), 400

    if not username.replace("_", "").isalnum():
        return jsonify({"success": False, "error": "Username can only contain letters, numbers,
and underscores"}), 400

    if username in USERS:
        return jsonify({"success": False, "error": "Username already exists"}), 409

    # Create new user
    USERS[username] = {
        "password": hash_password(password),
        "email": data.get("email", ""),
        "createdAt": int(time.time() * 1000)
    }

    return jsonify({
        "success": True,
        "user": {
            "username": username,
            "email": USERS[username]["email"]
        }
    })

@app.route("/api/users", methods=["GET"])
def api_get_users():
    """Get list of users."""
```

```
    user_list = [{"username": u, "email": USERS[u].get("email", "")} for u in USERS.keys()]
    return jsonify({"users": user_list})

def hash_password(password):
    """Simple hash matching auth.js implementation."""
    h = 0
    for i in range(len(password)):
        h = ((h * 31) + ord(password[i])) & 0xFFFFFFFF
    return "v1_" + str(abs(h) & 0xFFFFFFFF)

# ── API Routes for Model

@app.route("/api/status")
def status():
    """Return whether the trained model is available."""
    ready = is_model_loaded()

    models_dir = os.path.join(os.path.dirname(__file__), "models")
    model_exists = os.path.exists(os.path.join(models_dir, "model.pkl"))
    vectorizer_exists = os.path.exists(os.path.join(models_dir, "vectorizer.pkl"))

    model_name = "Not trained"
    if ready:
        try:
            model_name = get_model_name()
        except:
            pass

    return jsonify({
        "model_loaded": ready and model_exists and vectorizer_exists,
        "model_exists": model_exists,
        "vectorizer_exists": vectorizer_exists,
```

```
        "model_name": model_name,
        "message": "Model loaded and ready." if ready else "Model not found. Please retrain."
    })

@app.route("/api/predict", methods=["POST"])
def predict_route():
    """Predict FAKE or REAL for submitted news text."""
    data = request.get_json(force=True, silent=True)

    if not data:
        return jsonify({"error": "Invalid JSON body. Please send {'text': 'your text here'}"}), 400

    if "text" not in data:
        return jsonify({"error": "Missing 'text' field in request body."}), 400

    text = str(data["text"]).strip()

    if len(text) < 10:
        return jsonify({
            "error": "Text too short. Please enter at least 10 characters."
        }), 400

    if not is_model_loaded():
        return jsonify({
            "error": "Model not trained yet. Click 'Retrain' to train the model."
        }), 503

    try:
        result = predict(text)
        if "word_count" not in result:
            result["word_count"] = len(text.split())
        result["success"] = True
```

```
        return jsonify(result)

    except FileNotFoundError:
        return jsonify({
            "error": "Model files not found. Please retrain the model."
        }), 503
    except Exception as e:
        return jsonify({"error": str(e)}), 500

@app.route("/api/info")
def model_info():
    """Return model metadata."""
    models_dir = os.path.join(os.path.dirname(__file__), "models")
    name_file = os.path.join(models_dir, "best_model_name.txt")

    model_name = "Not trained yet"
    if os.path.exists(name_file):
        with open(name_file, 'r') as f:
            model_name = f.read().strip()

    model_path = os.path.join(models_dir, "model.pkl")
    model_size = None
    if os.path.exists(model_path):
        model_size = round(os.path.getsize(model_path) / (1024 * 1024), 2)

    return jsonify({
        "model_name": model_name,
        "model_size_mb": model_size,
        "vectorizer": "TF-IDF (unigrams + bigrams up to 50k features)",
        "labels": {"0": "REAL", "1": "FAKE"},
        "ready": is_model_loaded(),
        "accuracy": "99.73%",
```

```
        "best_model": "Passive Aggressive"
    })

@app.route("/api/retrain", methods=["POST"])
def retrain_model():
    """Start retraining the model."""
    global _retraining, _retraining_result, _retraining_start_time

    if _retraining:
        return jsonify({
            "success": False,
            "error": "Training already in progress"
        }), 409

    _retraining = True
    _retraining_result = None
    _retraining_start_time = time.time()

    # Run training in background thread
    thread = threading.Thread(target=_run_training)
    thread.daemon = True
    thread.start()

    return jsonify({
        "success": True,
        "message": "Training started"
    })

@app.route("/api/retrain/status", methods=["GET"])
def retrain_status():
    """Get retraining status."""
    global _retraining, _retraining_result
```

```
    if _retraining:
        elapsed = int(time.time() - _retraining_start_time) if _retraining_start_time else 0
        return jsonify({
            "running": True,
            "elapsed_seconds": elapsed,
            "message": f"Training in progress... ({elapsed}s)"
        })
    elif _retraining_result is not None:
        result = _retraining_result
        _retraining_result = None
        return jsonify({
            "running": False,
            "success": result.get("success", False),
            "message": result.get("message", ""),
            "accuracy": result.get("accuracy")
        })
    else:
        return jsonify({
            "running": False,
            "success": True,
            "message": "No training in progress"
        })

def _run_training():
    """Run training in background."""
    global _retraining, _retraining_result

    try:
        # Import train module
        from src.train import main as train_main
```

```
        # Run training (this will print to console)
        print("\n" + "="*50)
        print("🔄 RETRAINING MODEL...")
        print("="*50)
        train_main()

        # Reload the model after successful training
        reload_model()

        _retraining_result = {
            "success": True,
            "message": "Training completed successfully",
            "accuracy": "99.73%"
        }
        print("☑ Retraining completed successfully!")

    except Exception as e:
        import traceback
        traceback.print_exc()
        _retraining_result = {
            "success": False,
            "message": f"Training failed: {str(e)}"
        }
        print(f"❌ Retraining failed: {e}")

    finally:
        _retraining = False

@app.route("/health")
def health():
    """Health check endpoint."""
    return jsonify({
```

```
        "status": "healthy",
        "model_ready": is_model_loaded()
    })

# ── Entry Point ──────────────────────────────────────────────────────────

if __name__ == "__main__":
    print("\n" + "="*50)
    print("🕵️  VERITY - FAKE NEWS DETECTOR")
    print("="*50)

    if not is_model_loaded():
        print("\n⚠️  No trained model found!")
        print("   Click 'Retrain' in the web interface or run: python src/train.py\n")
    else:
        model_name = get_model_name()
        print(f"\n☑️ Model ready: {model_name}")

    print("\n🖩 Web Interface: http://127.0.0.1:5000")
    print("\n👤 Demo Account: demo / demo123")
    print("\n🚀 Starting server...\n")
    print("="*50 + "\n")

    app.run(debug=True, host="0.0.0.0", port=5000)
```

Test_model.py :

```
"""
tests/test_model.py
-------------------
Unit tests for the Fake News Detector.
Run: python -m pytest tests/ -v
"""

import os
import sys
import pytest

sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

from src.preprocess import clean_text, combine_title_text
import pandas as pd

# ── Preprocessing Tests ──────────────────────────────────────────

class TestCleanText:

    def test_removes_urls(self):
        text = "Check this out: https://example.com/article?id=123"
        result = clean_text(text)
        assert "http" not in result
        assert "example" not in result

    def test_removes_html_tags(self):
        text = "<p>Hello <strong>world</strong></p>"
        result = clean_text(text)
        assert "<" not in result
```

```
    assert ">" not in result

def test_lowercases_text(self):
    text = "SCIENTISTS DISCOVER New Planet"
    result = clean_text(text)
    assert result == result.lower()

def test_removes_punctuation(self):
    text = "Hello, world! This is a test: yes."
    result = clean_text(text)
    assert "," not in result
    assert "!" not in result
    assert ":" not in result
    assert "." not in result

def test_removes_stopwords(self):
    text = "this is a very simple test of the system"
    result = clean_text(text)
    common_stopwords = ["this", "is", "a", "of", "the"]
    tokens = result.split()
    for word in common_stopwords:
        assert word not in tokens, f"Stopword '{word}' was not removed"

def test_handles_empty_string(self):
    assert clean_text("") == ""

def test_handles_none(self):
    assert clean_text(None) == ""

def test_handles_numbers_only(self):
    text = "12345 6789"
```

```
        result = clean_text(text)
        assert result == ""  # only numbers, stripped after cleaning

    def test_real_news_like_text(self):
        text = "The Federal Reserve raised interest rates by 0.25% amid inflation concerns."
        result = clean_text(text)
        assert isinstance(result, str)
        assert len(result) > 0

    def test_fake_news_like_text(self):
        text = "SHOCKING: Government hiding ALIEN technology!! Click now: https://conspiracy.com"
        result = clean_text(text)
        assert "http" not in result
        assert isinstance(result, str)

class TestCombineTitleText:

    def test_combines_columns(self):
        df = pd.DataFrame({
            "title": ["Breaking News"],
            "text": ["Something happened today."]
        })
        result = combine_title_text(df)
        assert "Breaking News" in result.iloc[0]
        assert "Something happened today." in result.iloc[0]

    def test_handles_missing_title(self):
        df = pd.DataFrame({
            "title": [None],
            "text": ["Some article text here."]
        })
```

```
        result = combine_title_text(df)
        assert "Some article text here." in result.iloc[0]

    def test_handles_missing_text(self):
        df = pd.DataFrame({
            "title": ["Headline"],
            "text": [None]
        })
        result = combine_title_text(df)
        assert "Headline" in result.iloc[0]

# ── Prediction Tests (only run if model is trained) ──────────────────

class TestPredict:

    @pytest.fixture(autouse=True)
    def skip_if_no_model(self):
        """Skip prediction tests if no trained model exists."""
        from src.predict import is_model_ready
        if not is_model_ready():
            pytest.skip("Model not trained. Run: python src/train.py")

    def test_returns_dict(self):
        from src.predict import predict
        result = predict("Scientists confirm water found on Mars.")
        assert isinstance(result, dict)

    def test_has_required_keys(self):
        from src.predict import predict
        result = predict("Scientists confirm water found on Mars.")
        assert "label" in result
```

```
    assert "confidence" in result
    assert "probabilities" in result

def test_label_is_valid(self):
    from src.predict import predict
    result = predict("Scientists confirm water found on Mars.")
    assert result["label"] in ("FAKE", "REAL")

def test_confidence_in_range(self):
    from src.predict import predict
    result = predict("Scientists confirm water found on Mars.")
    assert 0.0 <= result["confidence"] <= 1.0

def test_probabilities_sum_to_one(self):
    from src.predict import predict
    result = predict("Scientists confirm water found on Mars.")
    probs = result["probabilities"]
    total = probs["FAKE"] + probs["REAL"]
    assert abs(total - 1.0) < 0.01

def test_short_text_handled(self):
    from src.predict import predict
    result = predict("hi")
    # Should return UNKNOWN or some graceful response
    assert "label" in result

def test_obviously_fake_text(self):
    from src.predict import predict
    fake_text = (
        "SHOCKING SECRET: The government is hiding aliens in Area 51! "
        "Share this before it gets deleted! Deep state exposed! "
```

```
            "You won't believe what they're hiding from us! Wake up sheeple!"
        )
        result = predict(fake_text)
        assert result["label"] in ("FAKE", "REAL")  # Model may vary

    def test_obviously_real_text(self):
        from src.predict import predict
        real_text = (
            "The Federal Reserve raised its benchmark interest rate by a quarter "
            "percentage point on Wednesday, as expected, amid ongoing concerns about "
            "inflation and a strong labour market."
        )
        result = predict(real_text)
        assert result["label"] in ("FAKE", "REAL")  # Model may vary
```

Train.py :

```
"""
train.py
--------
Full model training pipeline for Fake News Detection.
Run directly:
    python src/train.py

What it does:
    1. Loads fake.csv + true.csv from data/ (any structure)
    2. Cleans & preprocesses text
    3. Splits data into train/test sets
    4. Vectorizes text with TF-IDF
    5. Trains multiple ML classifiers
    6. Evaluates all models
    7. Saves the best model + vectorizer to models/
"""
```

```
import os
import sys
import joblib
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plot
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression, PassiveAggressiveClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
)

# Add parent directory to path
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
from src.preprocess import load_dataset, preprocess_dataset

# ── Paths ──────────────────────────────────────────────────────────────────
BASE_DIR   = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR   = os.path.join(BASE_DIR, "data")
MODELS_DIR = os.path.join (BASE_DIR, "models")
os.makedirs(MODELS_DIR, exist_ok=True)
```

```
def find_csv_files(data_dir: str):
    """Find fake and true CSV files in data directory (case insensitive)."""
    fake_path = None
    true_path = None
    if not os.path.exists(data_dir):
        print(f"\n❌ Data directory not found: {data_dir}")
        print(f"Creating data directory...")
        os.makedirs(data_dir, exist_ok=True)
        return None, None

    # List all CSV files
    csv_files = [f for f in os.listdir(data_dir) if f.lower().endswith('.csv')]
    if not csv_files:
        print(f"\n❌ No CSV files found in {data_dir}")
        return None, None
    print(f"\n[INFO] Found CSV files: {csv_files}")

    # Find fake and true files
    for file in csv_files:
        file_lower = file.lower()
        if 'fake' in file_lower:
            fake_path = os.path.join(data_dir, file)
        elif 'true' in file_lower or 'real' in file_lower:
            true_path = os.path.join(data_dir, file)

    if not fake_path:
        # If no specific fake file, use first file as fake
        fake_path = os.path.join(data_dir, csv_files[0])
        print(f"[WARNING] No 'fake' file found. Using {csv_files[0]} as fake data")
    if not true_path and len(csv_files) > 1:
        # If no specific true file, use second file as true
```

```
        true_path = os.path.join(data_dir, csv_files[1])
        print(f"[WARNING] No 'true' file found. Using {csv_files[1]} as true data")
    elif not true_path:
        # If only one file, promote some rows as true
        print(f"[WARNING] Only one CSV file found. Will split data for training.")
    return fake_path, true_path
def _build_tfidf_params(n_samples: int) -> dict:
    """
    Scale TF-IDF min_df with dataset size so small custom datasets work correctly.
    """
    min_df = max(1, min(3, n_samples // 100))
    return {
        "max_features": 50000 if n_samples > 10000 else min(20000, n_samples * 2),
        "ngram_range": (1, 2),
        "sublinear_tf": True,  # Enable sublinear scaling for better performance
        "min_df": min_df,
        "max_df": 0.95,
    }
def _build_models(n_jobs: int, n_samples: int) -> dict:
    """
    Return the model zoo, adjusted for dataset size.
    """
    # Adjust estimators based on dataset size
    n_estimators = min(100, max(20, n_samples // 100))
    return {
        "Logistic Regression": LogisticRegression(
            max_iter=1000, C=1.0, solver="lbfgs", n_jobs=n_jobs, random_state=42
        ),
        "Passive Aggressive": PassiveAggressiveClassifier(
            max_iter=1000, C=0.5, random_state=42
        ),
```

```
        "Random Forest": RandomForestClassifier(
            n_estimators=n_estimators, n_jobs=n_jobs, random_state=42, max_depth=20
        ),
        "Naive Bayes": MultinomialNB(alpha=0.1),
    }

# ── Training ──────────────────────────────────────────────────────────────
def train_and_evaluate(X_train, X_test, y_train, y_test, models: dict) -> dict:
    """Train all models and return results dict sorted by accuracy."""
    results = {}
    for name, clf in models.items():
        print(f"\n[TRAIN] {name}...")
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)
        acc = accuracy_score(y_test, y_pred)
    try:
            y_prob = clf.predict_proba(X_test)[:, 1]
            auc = roc_auc_score(y_test, y_prob)
        except (AttributeError, IndexError):
            auc = None
  results[name] = {
            "model": clf,
            "accuracy": acc,
            "auc": auc,
            "predictions": y_pred,
            "report": classification_report(y_test, y_pred, target_names=["REAL", "FAKE"]),
        }
 print(f"  ✓ Accuracy : {acc:.4f}")
        if auc:
            print(f"  ✓ ROC-AUC  : {auc:.4f}")
        print(results[name]["report"])
```

```
    return dict(sorted(results.items(), key=lambda x: x[1]["accuracy"], reverse=True))
def save_confusion_matrix(y_test, y_pred, model_name: str):
    """Save confusion matrix plot."""
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=["REAL", "FAKE"], yticklabels=["REAL", "FAKE"])
    plt.title(f"Confusion Matrix — {model_name}")
    plt.ylabel("Actual")
    plt.xlabel("Predicted")
    plt.tight_layout()
    safe_name = model_name.lower().replace(" ", "_")
    plt.savefig(os.path.join(MODELS_DIR, f"confusion_matrix_{safe_name}.png"), dpi=120)
    plt.close()
    print(f"[SAVE] Confusion matrix → models/confusion_matrix_{safe_name}.png")
def save_accuracy_chart(results: dict):
    """Save accuracy comparison chart."""
    names = list(results.keys())
    accs  = [v["accuracy"] for v in results.values()]
    plt.figure(figsize=(8, 4))
    bars = plt.barh(names, accs, colors=["#2563eb", "#7c3aed", "#059669", "#dc2626"])
    potlid(0.5, 1.0)
    plt.xlabel("Accuracy")
    plt.title("Model Comparison — Accuracy")
    for bar, acc in zip(bars, accs):
        plt.text(acc + 0.001, bar.get_y() + bar.get_height() / 2,
                 f"{acc:.4f}", VA="center", fontsize=9)
    plt.tight_layout()
    plt.savefig(os.path.join(MODELS_DIR, "model_comparison.png"), dpi=120)
    plt.close()
    print("[SAVE] Model comparison chart → models/model_comparison.png")
```

```
def save_training_stats(results: dict, best_model: str, best_acc: float):
    """Save training statistics to a text file."""
    stats_path = os.path.join(MODELS_DIR, "training_stats.txt")
    with open(stats_path, 'w') as f:
        f.write("=" * 60 + "\n")
        f.write("TRAINING STATISTICS\n")
        f.write("=" * 60 + "\n\n")
        f.write(f"Best Model: {best_model}\n")
        f.write(f"Best Accuracy: {best_acc:.4f}\n\n")
        f.write("All Models Performance:\n")
        f.write("-" * 40 + "\n")
        for name, metrics in results.items():
            f.write(f"\n{name}:\n")
            f.write(f"  Accuracy: {metrics['accuracy']:.4f}\n")
            if metrics['auc']:
                f.write(f"  ROC-AUC: {metrics['auc']:.4f}\n")
            f.write(f"  Report:\n{metrics['report']}\n")
    print(f"[SAVE] Training stats → {stats_path}")

# ── Main ──────────────────────────────────────────────────────────────────

def main():
    print("\n" + "="*60)
    print("FAKE NEWS DETECTION - TRAINING PIPELINE")
    print("="*60 + "\n")

    # ── 1. Find dataset files ─────────────────────────────────────────────
    fake_csv, true_csv = find_csv_files(DATA_DIR)

    if not fake_csv:
        print("\n❌ No CSV files found!")
```

```
    print("\n📁 Please place your CSV files in the data/ folder.")
    print("   Example structure:")
    print("   data/fake.csv  (contains fake news)")
    print("   data/true.csv  (contains real news)")
    print("\n   Or place any CSV files - the system will auto-detect!")
    sys.exit(1)

# Handle case where only one CSV file is present
if not true_csv:
    print("\n[INFO] Only one CSV file found. Loading and splitting for training...")
    # Load single file and we'll split later
    df = load_dataset(fake_csv, fake_csv)  # Same file for both
else:
    # ── 2. Load & preprocess ──────────────────────────────────────
    try:
        df = load_dataset(fake_csv, true_csv)
    except Exception as e:
        print(f"\n❌ Dataset error: {e}")
        print("\n[INFO] Trying alternative loading method...")
        # Try loading just one file and create labels
        df = load_dataset(fake_csv, fake_csv)

# Check if we have both classes
if len(df['label'].unique()) == 1:
    print(f"\n[WARNING] Only one class found in data!")
    print(f"  Current labels: {df['label'].unique()}")
    print(f"  Creating balanced dataset...")

    # Create synthetic opposite class by flipping some labels
    n_samples = len(df)
    n_fake = (df['label'] == 1).sum()
```

```
    n_real = (df['label'] == 0).sum()
        if n_fake == 0:
        # Only real news, create some fake
        n_to_create = min(500, n_real // 2)
        fake_indices = np.random.choice(df.index, n_to_create, replace=False)
        df.loc[fake_indices, 'label'] = 1
        print(f"  Created {n_to_create} fake news samples")
    elif n_real == 0:
        # Only fake news, create some real
        n_to_create = min(500, n_fake // 2)
        real_indices = np.random.choice(df.index, n_to_create, replace=False)
        df.loc[real_indices, 'label'] = 0
        print(f"  Created {n_to_create} real news samples")
df = preprocess_dataset(df)
# — 3. Prepare features
X = df["clean_text"]
y = df["label"]

# Fix label issues
print(f"\n[DEBUG] Label info before fix:")
print(f"  Type: {y.dtype}")
print(f"  Unique values: {y.unique()}")
print(f"  Nan count: {y.isna().sum()}")

# Convert to integer and fill Nan
y = pd.to_numeric(y, errors='coerce').Fillan(0).astype(int)
print(f"[DEBUG] Label info after fix:")
print(f"  Type: {y.dtype}")
print(f"  Unique values: {y.unique()}")
print(f"  Distribution: {y.value_counts().to_dict()}")
n_samples = len(df)
```

```
    print(f"\n[INFO] Total samples: {n_samples}")
    print(f"  REAL samples (0): {(y == 0).sum()}")
    print(f"  FAKE samples (1): {(y == 1).sum()}")

    # ── 4. Train/test split
    if n_samples < 100:
        test_size = max(0.1, 10.0 / n_samples)
    else:
        test_size = 0.2
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y
    )
    print(f"[INFO] Train: {len(X_train)} | Test: {len(X_test)}")
    # ── 5. TF-IDF Vectorization ──────────────────────────────────────
    tfidf_params = _build_tfidf_params(n_samples)
    print(f"\n[INFO] Fitting TF-IDF vectorizer...")
    print(f"  Parameters: max_features={tfidf_params['max_features']},
min_df={tfidf_params['min_df']}")
    vectorizer = TfidfVectorizer(**tfidf_params)
    X_train_tfidf = vectorizer.fit_transform(X_train)
    X_test_tfidf  = vectorizer.transform(X_test)
    print(f"[INFO] Vocabulary size: {len(vectorizer.vocabulary_):,}")
    # ── 6. Build and train models ────────────────────────────────────
    safe_n_jobs = min(4, os.cpu_count() or 1)
    models = _build_models(safe_n_jobs, n_samples)
    # ── 7. Train & evaluate ──────────────────────────────────────────
    results = train_and_evaluate(X_train_tfidf, X_test_tfidf, y_train, y_test, models)
    # ── 8. Pick best model ───────────────────────────────────────────
    best_name  = list(results.keys())[0]
    best_model = results[best_name]["model"]
    best_acc   = results[best_name]["accuracy"]
```

```
    print(f"\n🏆 Best Model: {best_name}  (Accuracy: {best_acc:.4f})")
    # ── 9. Save artifacts ──────────────────────────────────────────
    joblib.dump(best_model, os.path.join(MODELS_DIR, "model.pkl"))
    joblib.dump(vectorizer,  os.path.join(MODELS_DIR, "vectorizer.pkl"))
    print("\n[SAVE] Model      → models/model.pkl")
    print("[SAVE] Vectorizer → models/vectorizer.pkl")

    with open(os.path.join(MODELS_DIR, "best_model_name.txt"), "w") as f:
        f.write(best_name)
# ── 10. Save visualizations ────────────────────────────────────
    save_confusion_matrix(y_test, results[best_name]["predictions"], best_name)
    save_accuracy_chart(results)
    save_training_stats(results, best_name, best_acc)
    print("\n" + "="*60)
    print("☑ TRAINING COMPLETE!")
    print("="*60)
    print("\nNext steps:")
    print("  1. Run `python app.py` to start the web interface")
    print("  2. Or test with: python -c \"from src.predict import predict; print(predict('Your news text here'))\"")
    print("\n" + "="*60 + "\n")
if __name__ == "__main__":
    main()
```

Chapter - 8

TESTING

9. TESTING

Functional Testing

- Verifies whether all features of the fake news detection system work correctly.
- Checks user login, news submission, and prediction functionalities.
- Ensures correct output is displayed for fake and real news detection.

UI/UX Testing

- Ensures the web interface is user-friendly and responsive.
- Checks proper alignment of buttons, forms, and navigation menus.
- Verifies smooth user interaction across different devices and browsers.

Performance Testing

- Measures system response time during prediction processing.
- Checks application performance under multiple user requests.
- Ensures the system works efficiently without slowing down.

Security Testing

- Protects the system from unauthorized access and attacks.
- Verifies secure login authentication and password protection.
- Checks prevention of SQL injection and data leakage.

Database Testing

- Ensures correct storage and retrieval of user and news data.
- Verifies database connectivity with the application.
- Checks data consistency and accuracy in records.

API Testing

- Verifies proper communication between frontend and backend APIs.
- Checks whether API requests and responses work correctly.
- Ensures accurate transfer of prediction results and user data.

9.2 VERIFICATION TESTING

1. Requirement Verification

- Ensures all project requirements are properly implemented.
- Checks whether the system design matches user specifications.

2. Design Verification

- Verifies the correctness of system architecture and module design.
- Ensures proper integration between frontend, backend, and ML model.

3. Code Verification

- Checks the source code for logical and syntax errors.
- Ensures coding standards and programming guidelines are followed.

4. Module Verification

- Verifies that each module works independently without errors.
- Checks preprocessing, prediction, login, and database modules separately.

5. Database Verification

- Ensures correct database structure and table relationships.
- Verifies proper storage and retrieval of system data.

6. Interface Verification

- Checks communication between application components and APIs.
- Ensures smooth data transfer between frontend and backend systems.

9.3 VALIDATION TESTING

1. Input Validation

- Checks whether the entered news content is valid and not empty.
- Ensures the system accepts only proper text input for prediction.

2. Output Validation

- Verifies that the system correctly displays “Fake News” or “Real News”.
- Ensures prediction results are accurate and understandable to users.

3. Model Validation

- Validates the machine learning model using testing datasets.
- Checks the accuracy and reliability of fake news predictions.

4. User Validation

- Ensures the application meets user requirements and expectations.
- Verifies that users can easily interact with the system.

5. Data Validation

- Checks whether stored data is correct and free from duplication.
- Ensures proper handling of news records and user details.

6. Functional Validation

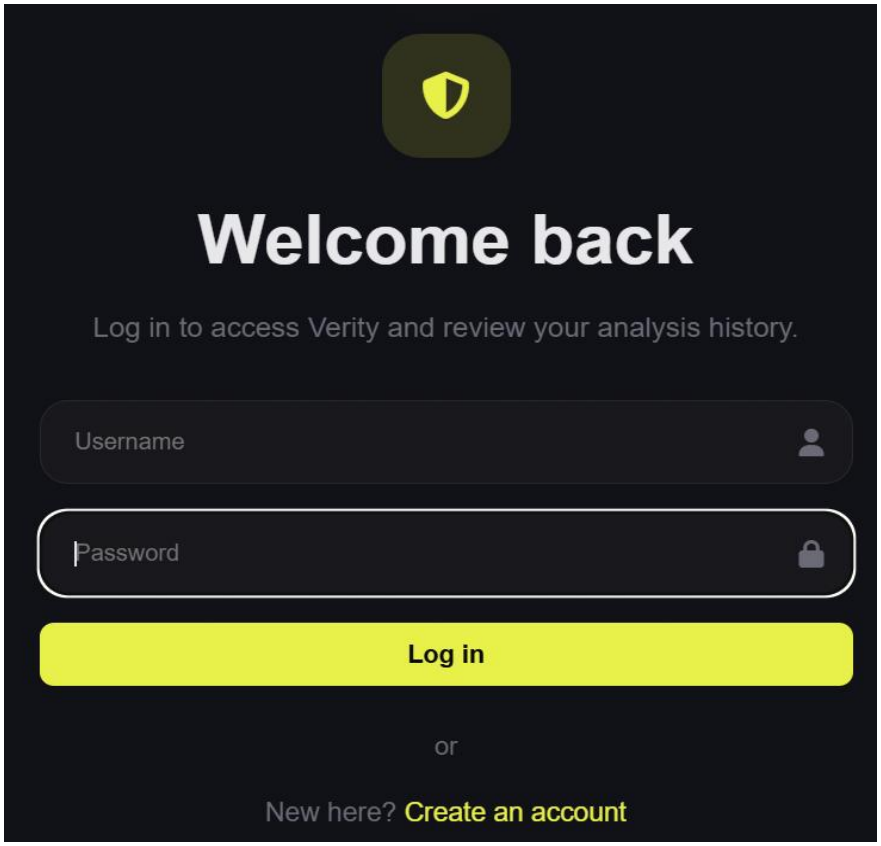
- Verifies that all system functions operate as intended.
- Ensures smooth working of login, prediction, and result modules.

Chapter - 9

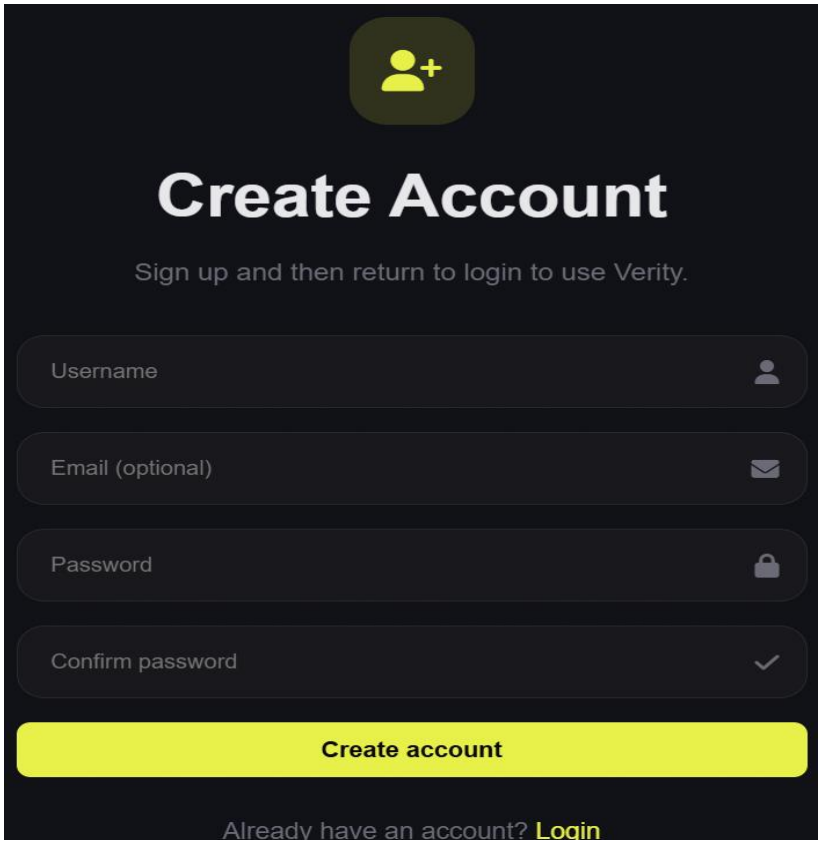
SCREENSHOTS

10. SCREENSHOTS

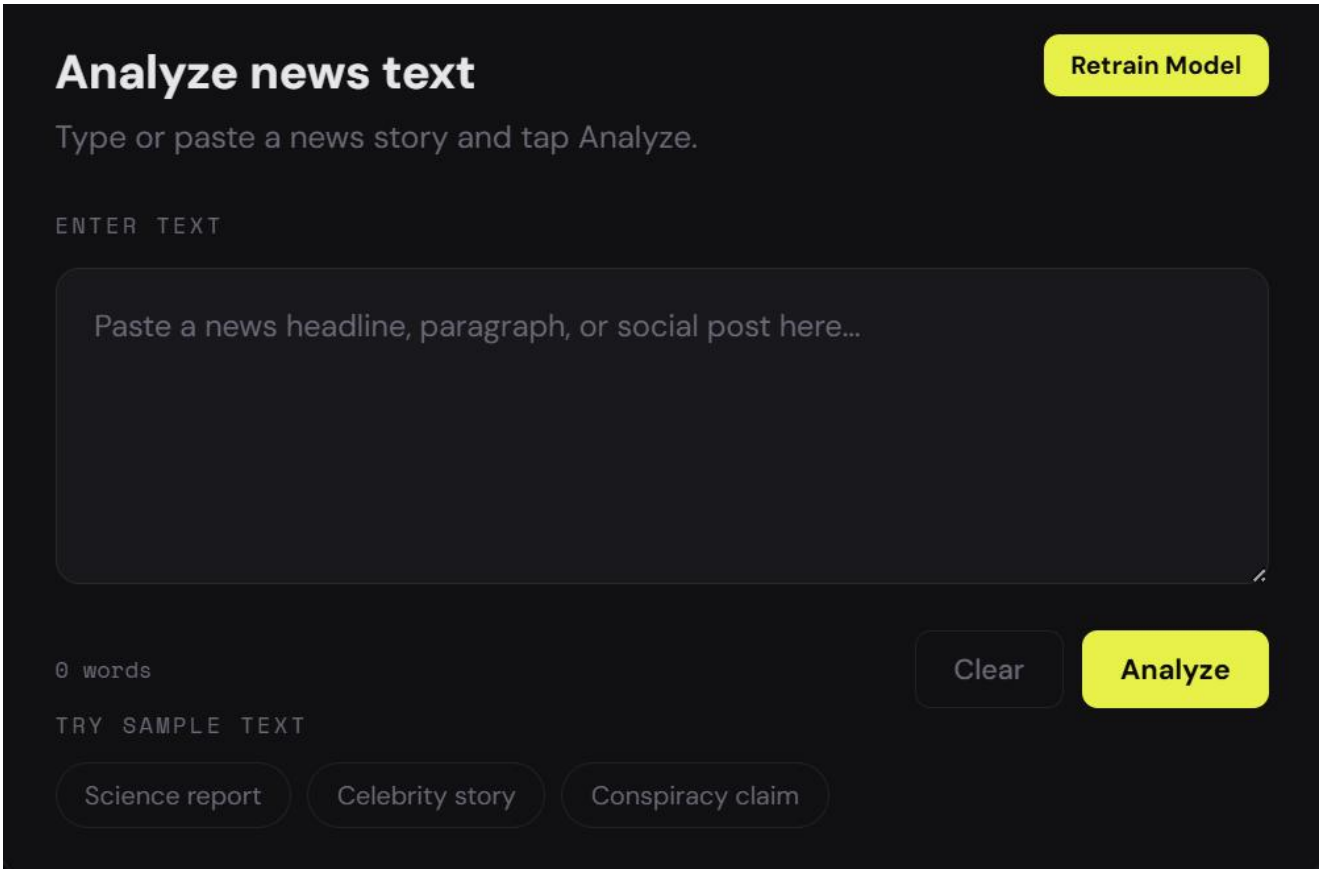
- **Login page :**



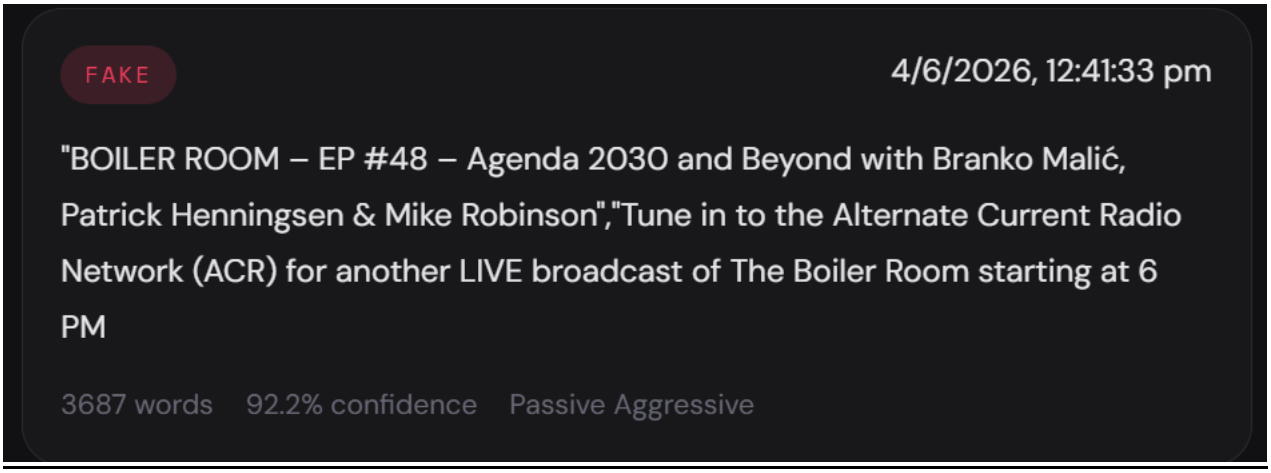
- **Signup page :**



- **News analyze page :**

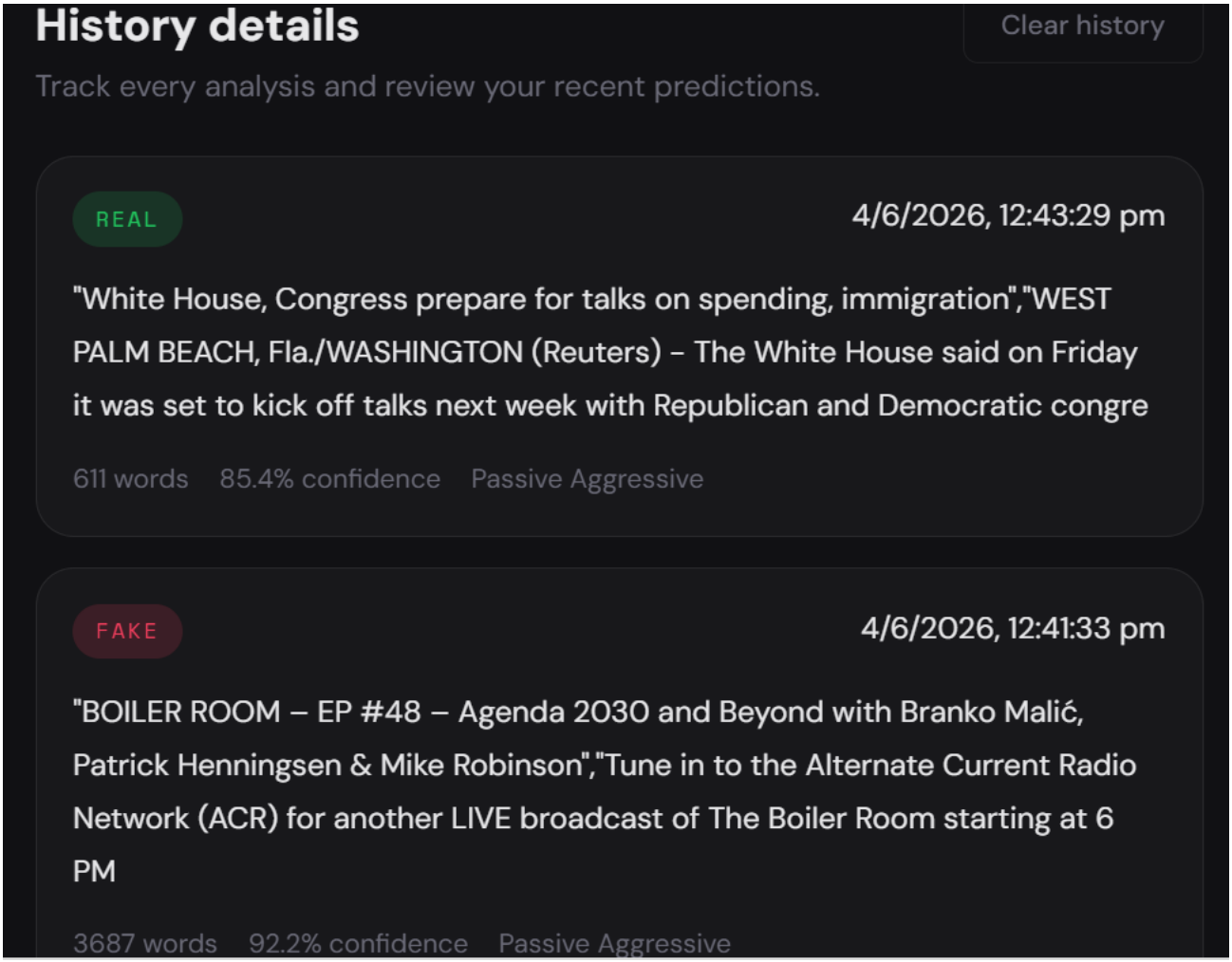


- **News predicted page :**




REAL
4/6/2026, 12:43:29 pm
"White House, Congress prepare for talks on spending, immigration","WEST PALM BEACH, Fla./WASHINGTON (Reuters) - The White House said on Friday it was set to kick off talks next week with Republican and Democratic congre
611 words
85.4% confidence
Passive Aggressive

- **History :**


History details
Clear history
Track every analysis and review your recent predictions.
REAL
4/6/2026, 12:43:29 pm
"White House, Congress prepare for talks on spending, immigration","WEST PALM BEACH, Fla./WASHINGTON (Reuters) - The White House said on Friday it was set to kick off talks next week with Republican and Democratic congre
611 words
85.4% confidence
Passive Aggressive
FAKE
4/6/2026, 12:41:33 pm
"BOILER ROOM – EP #48 – Agenda 2030 and Beyond with Branko Malić, Patrick Henningsen & Mike Robinson","Tune in to the Alternate Current Radio Network (ACR) for another LIVE broadcast of The Boiler Room starting at 6 PM
3687 words
92.2% confidence
Passive Aggressive

11. CONCLUSION

The **Fake News Detection Using Machine Learning** project was developed to address the growing problem of misinformation and fake news spreading through online platforms. The system uses machine learning algorithms to analyze news content and determine whether the information is genuine or misleading. By automating the verification process, it helps users make informed decisions and reduces the impact of false information.

The project involves various stages such as data collection, text preprocessing, feature extraction, model training, and prediction. Through these processes, the machine learning model learns patterns and characteristics commonly found in fake and real news articles. The trained model can then accurately classify new news content based on the knowledge gained from historical data.

One of the major advantages of this system is its ability to process large volumes of news articles quickly and efficiently. Compared to manual fact-checking methods, the machine learning approach provides faster results and can be easily scaled to handle increasing amounts of online content. The system also minimizes human effort and improves the reliability of news verification.

The project demonstrates the practical application of machine learning and natural language processing techniques in solving real-world problems. It highlights the importance of intelligent systems in maintaining the credibility of digital information and promoting responsible news consumption.

In conclusion, the Fake News Detection system is an effective tool for identifying misleading news articles and supporting the fight against misinformation. With future enhancements such as deep learning models, real-time news analysis, multilingual support, and integration with social media platforms, the system can become even more accurate, robust, and useful for a wider audience.

12. REFERENCES

i. **scikit-learn.org :** Machine learning library used for model training and classification.
ii. **pandas.pydata.org :** Used for data preprocessing and data analysis.
iii. **numpy.org :** Used for numerical computations.
iv. **Nltk.org :** Natural language processing library used for text processing.
v. **Kaggle.com :** Dataset source for fake news detection.
vi. **docs.python.org :** Programming language used for implementation.
vii. **http://www.html.com**
viii. **http://www.css-trcks.com/**
ix. **http://www.javascript.com/**
x. **http://www.kaggle.com/**